Agenda

- Understanding internals of SQL Query (Contd.)
  - JOINs
  - AGGR.
  - GROUP BY
  - ⋮
  - Order of SQL Query Execute
- Stored Procedures

⊗ Same query may follow diff. query plans depending on data size.



Index Seek v/s Index Scan

"Select * from emp where emp_id > 30"

↓ PK → clustered

Index Sc...

↓ PK → clustered index

(*) (1) Department

dep_id (PK), dept. name

(2) Employee

emp_id (PK), fName, lName, hire_date, salary, dept_id

(2020-02-01)

(3) Project

proj_id (PK), proj_name, manager_id

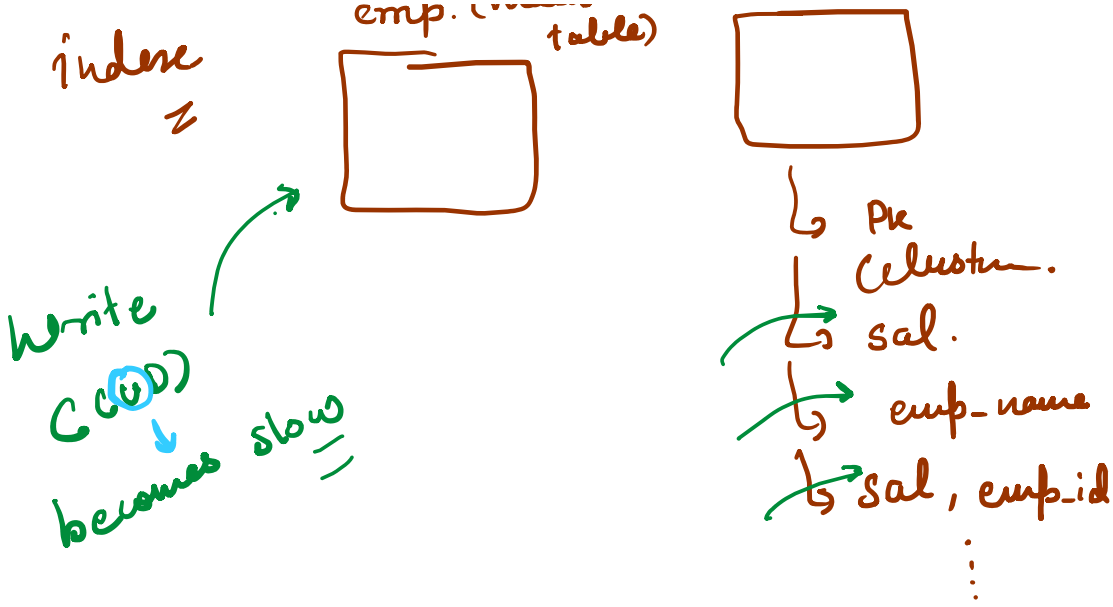
Q Can we create index on all cols. (for the ones used in across req.) ?

Ans: Ideally, Yes.

Recommended → NO

(DB) → R v/s W ratio.





If DB is Read heavy

↳ R:W > 50:1

50K req/min

↓

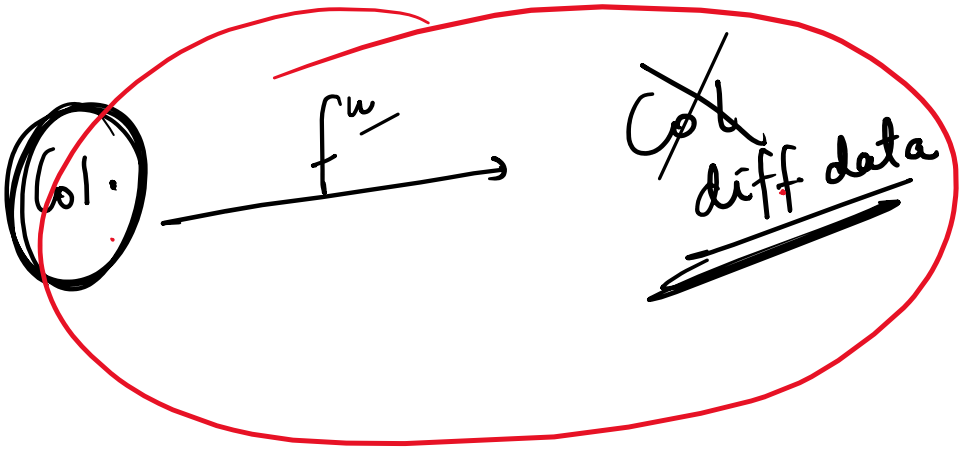
1K req/min ⇒ write

Q Select emp_id, name, hire_date
from employee
where YEAR (hire_date)
= 2020

No Indexes → Full Table Scan

~~No~~ Index → Full Table Scan

Index(Hire_date) →




```
Select ------
  from employees
    where
    hire_data >= '01-01-2020'
     and
     hire_date <= '31-12-2020';
```

col. → (address) string → index

```
Select ----
  from employees
    where address like "%Indraprastha%";
```

Full Text Index

(*) sort:

by sal;

{Select * from employees} → resultset

where - - - -

order by sal desc

sort()

in-Memory sorting

HEAP-SORT

sorting → complete data



resultset huge → In-mem sort won't work

creates a temp table in disk.

"External Merge Sort"

→ If index on Hire-date

Req. sort by Hire-date

Index Seek

* AGGREGATE fu:

count(), sum(), max(), min(),

avg(), count(), sum(), max(), min(), ...

```
count=0
sum=0
for(....) {
    sum += emp["salary"];
    count ++;
}
avg = sum/count;
```

Full table scan

index

⊛ GROUP BY

Q. Calculate the avg. salary of every dept. ??

employees

name	dept	sal
E1	Sales	30k
E2	Dev	40k
E3	Sales	60k
E4	Sales	30k
E5	Exec	200k
E6	Dev	80k
E7	Dev	120k

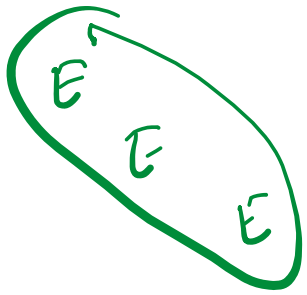
120/3

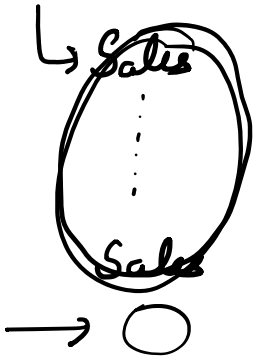
Sales → 60k

Dev → 100k

Exec → 250k







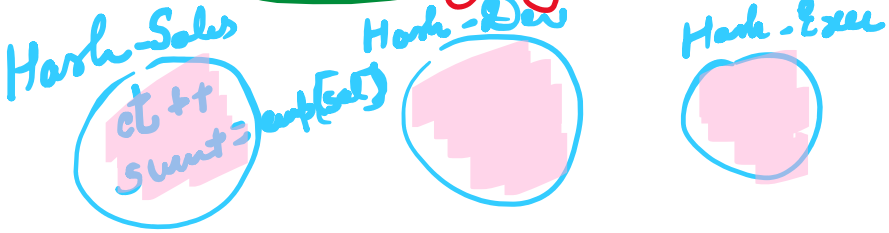

```
Select avg(salary)
from employees
where dept = 'Sales'
```

```
Select
    avg(salary), dept_id
    from employees
    group by dept_id.;
```

→ Full table Scan

+

Hash-Aggregation Method



for ( ) {

}

DS

~~Select~~ give me sum of all unique values

[35, 11, 11, 35, 11, 35, 35,

35 → 140
11 → 33
⋮

prereq
↳ sorted.



9f dept_id → indexed

↓

Full Table Scan

Full Table Scan

index Table

no Hash-Aggregation

HAVING

where → complete table,

Select avg. sal of all dept.
where avg. sal > 10000.

group-by → First col. avg. sal. of every dept.

every dept.

↳ Filter for given condition

Group-by + Filtering on grouped data

↳ HAVING

```
Select
    avg(salary) As avg_sal
    ,dept_id
from employees
group by dept_id
Having avg_sal > 100k;
```

Filtering

Filtering

(*) Diff. b/w where & Having

minute

where → eliminate row only

Having → group by

ⓐ JOINs

Inner Join (Intersection)

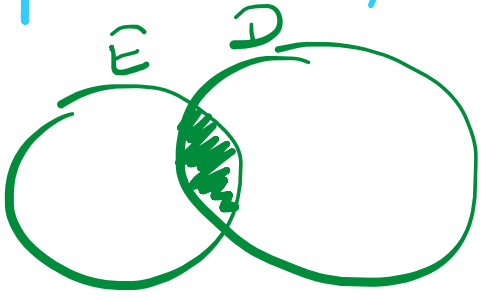
employee

name	dept_id
E1	1
E2	2
E3	1
E4	4
E5	2
E6	null

departments

dept_id	name
1	- -
2	- -
3 ?	- -
4	- -

e.deptid = d.dept_id



? How DB will do/execute Joins ?

Select e.name, d.name

from
employees e
Inner Join
departments d
~~using (dept_id)~~ on ------.

Option 1

① Use Nested loop

#employees → E
#dept → D

emp / dept

```
for ( record1 in table1) {
    for (record2 in table2) {
        if( <JoinCond> ) {
            // result set prepared
        }
    }
}
```

dept / emp.

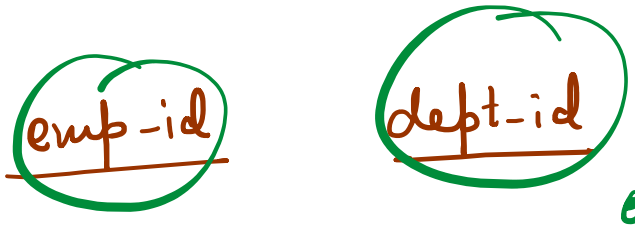
$T(N) = O(E * D)$

Index

↳ clustered-index of dept_id

↳ $O(E * \log D)$

↳ O(E * log D)



or

dept-id → clustered index (dept table)

dept-id → non-clustered index (emp table)

outer table

↳ for( ) {

inner table

↳ for( ) {

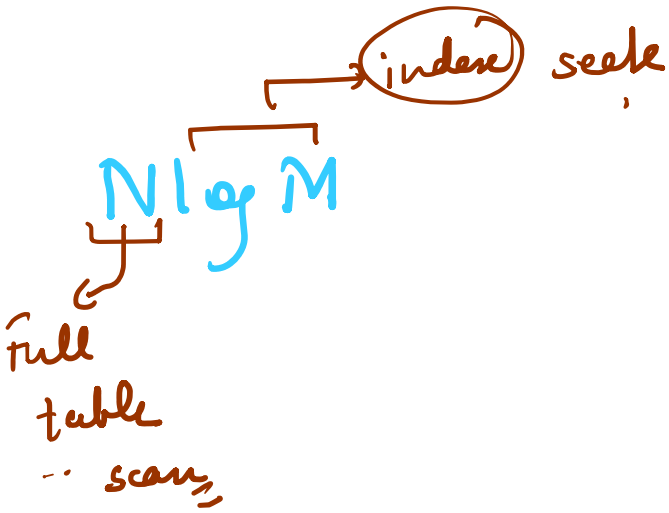
} }

outer table	inner table	time-comp
emp	dept	E log D
dept	emp	D log E

large table

↳ employee

↳ department


index seek
N log M
full
table
scan